Graph Representations (Adjacency Matrix & Adjacency List) Data Structures

> **Definition:** Graph representations are memory data structures used to store the vertices and edge relationships of a graph in computer memory. The two primary methods are **Adjacency Matrix** and **Adjacency List**.

---

1. Detailed Technical Explanation

```

Sampl

### 1. Adjacency Matrix Representation

A 2D array `adj[V][V]` of size **V  V**, where `adj[i][j] = 1` if an edge exists from vertex `i` to vertex `j`, and `0` otherwise.

#### Adjacency Matrix for Sample Graph:

```

---

2. Adjacency List Representation

An array of linked lists `adj[V]`, where each element `adj[i]` points to a linked list of neighboring vertices connected to vertex `i`.

Adjacency List for Sample Graph:

```

---

## 2. Complete Comparison Table
```

| Feature | Adjacency Matrix | Adjacency List |
| :--- | :

---

3. C Implementation of Adjacency List

```c
#inclu

struct AdjListNode {
int des

struct Graph {
int V;

struct Graph* createGraph(int V) {
struct

void addEdge(struct Graph* graph, int src, int dest) {
// Add

// For undirected graph, add dest to src
newNo
```

---

4. Must-Write Points for Exams

- In an undirected graph's adjacency matrix, the matrix is always **symmetric** about the main diagonal (`adj[i][j] == adj[j][i]`).

- Adj

---

5. Quick Recall Flow

```

Adj Ma
```